

Lab Assignment No.	06
Title	Implementation of Clonal selection algorithm using Python.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 06

Title: Implementation of Clonal Selection Algorithm Using Python

Problem Statement: To design and implement the Clonal Selection Algorithm (CSA) using Python in Jupyter Notebook to optimize problem-solving capabilities inspired by the natural immune system.

Prerequisites:

- Basic knowledge of Python programming
- Understanding of optimization algorithms
- Familiarity with bio-inspired algorithms
- Basic knowledge of Jupyter Notebook environment

Software Requirements:

- Jupyter Notebook
- Python 3.x

Hardware Requirements:

- A computer with minimum 4 GB RAM
- Dual-core processor or higher
- Stable internet connection

Theory: The Clonal Selection Algorithm (CSA) is a bio-inspired optimization technique based on the clonal selection principle of the adaptive immune system. It is used to solve complex optimization problems by mimicking the process of natural selection, cloning, and mutation of immune cells.

Key Concepts:

1. **Antigen Recognition:** Identifying candidate solutions (antibodies) that can respond to the problem (antigen).
2. **Cloning:** Creating multiple copies of high-affinity antibodies.

3. **Mutation:** Introducing variations in cloned antibodies to explore new solutions.
4. **Selection:** Retaining the best solutions based on affinity or fitness.

The Clonal Selection Algorithm operates in an iterative manner, improving the solution population over successive generations. The fundamental processes are inspired by the natural immune response mechanism, which efficiently identifies and eliminates pathogens.

1. **Affinity Maturation:** The algorithm continuously improves the quality of solutions through the process of affinity maturation, where high-affinity antibodies are selected, cloned, and subjected to hypermutation to enhance their performance.
2. **Hypermutation:** This process introduces diversity into the population by applying mutations at higher rates, especially to low-affinity antibodies, allowing the exploration of new regions in the solution space.
3. **Replacement Strategy:** CSA employs a replacement mechanism where poorly performing antibodies are eliminated and replaced with new randomly generated antibodies. This prevents premature convergence and maintains population diversity.
4. **Convergence Criteria:** The algorithm terminates when a stopping condition is met, such as reaching a maximum number of generations or achieving a satisfactory fitness level.

Mathematical Model: The mathematical representation of CSA involves:

- **Affinity Calculation:** , where is the fitness function to be minimized.
- **Selection Probability:** Proportional to the affinity score, determining the likelihood of an antibody being cloned.
- **Mutation Rate:** Inversely proportional to the affinity, i.e., higher affinity leads to lower mutation rates and vice versa.

Applications of CSA:

- Optimization problems
- Pattern recognition
- Machine learning
- Data mining
- Network security
- Robotics path planning

Source Code –

```
# Import Required Libraries
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Define the Objective Function
```

```
def objective_function(x):
```

```
    return x**2 + 3*x + 5
```

```
# Initialize Population
```

```
def initialize_population(size, lower_bound, upper_bound):
```

```
    return np.random.uniform(lower_bound, upper_bound, size)
```

```
# Evaluate Affinity (Fitness)
```

```
def evaluate_fitness(population):
```

```
    return 1 / (1 + objective_function(population))
```

```
# Select Best Antibodies
```

```
def select_best(population, fitness, num_best):
```

```
    sorted_indices = np.argsort(-fitness)
```

```
    return population[sorted_indices[:num_best]]
```

```
# Clone Selected Antibodies
```

```
def clone_population(best_population, num_clones):
```

```
    clones = np.repeat(best_population, num_clones)
```

```
    return clones
```

```
# Hypermutation (Random Mutation)
```

```
def mutate(clones, mutation_rate):
```

```
    mutation = np.random.uniform(-mutation_rate, mutation_rate, size=clones.shape)
```

```
    return clones + mutation
```

```
# Replace Worst Solutions
```

```
def replace_worst(population,new_population, num_replace):
```

```
    population[-num_replace:] = new_population[:num_replace]
```

```
    return population
```

```
# Implement the Clonal Selection Algorithm
```

```
def clonal_selection_algorithm(pop_size=20, generations=50,num_best=5,num_clones=3,  
    mutation_rate=0.1):
```

```
    lower_bound, upper_bound = -10, 10
```

```
    population = initialize_population(pop_size, lower_bound, upper_bound)
```

```
    best_fitness_history = []
```

```
    for generation in range(generations):
```

```
        fitness = evaluate_fitness(population)
```

```
        best_population = select_best(population, fitness, num_best)
```

```
        clones = clone_population(best_population, num_clones)
```

```
        mutated_clones = mutate(clones, mutation_rate)
```

```
        fitness_mutated = evaluate_fitness(mutated_clones)
```

```
        best_mutated = select_best(mutated_clones, fitness_mutated, num_best)
```

```
        population = replace_worst(population, best_mutated, num_best)
```

```
        best_fitness_history.append(np.max(fitness))
```

```
        if generation % 10 == 0:
```

```
            print(f"Generation {generation}: Best fitness = {np.max(fitness)}")
```

```
    return best_fitness_history, population
```

```
# Run the Algorithm
```

```
best_fitness, final_population = clonal_selection_algorithm()
```

```
# Plot the Results
```

```
plt.plot(best_fitness)
```

```
plt.xlabel('Generations')  
plt.ylabel('Best Fitness')  
plt.title('Clonal Selection Algorithm Performance')  
plt.show()
```

Conclusion - Through this practical, we have explored the Clonal Selection Algorithm and its role in solving optimization problems inspired by the natural immune system. This practical highlighted the efficiency of bio-inspired algorithms in addressing complex problem-solving tasks, demonstrating the algorithm's ability to find optimal solutions through adaptive processes.